

Correction détaillée — La boucle « while » (2)

Exercice 1 — Somme des carrés

```

1 n = 1
2 s = 0
3 while s < 1000:
4     s = s + n**2
5     n = n + 1
6 print(n)

```

- **1. Utilité de n.** La variable **n** joue un **double rôle** : c'est à la fois l'entier dont on ajoute le carré (n^2) et le **compteur** qui avance de 1 en 1 (1, 2, 3, ...) pour parcourir successivement les carrés. Sans elle, on ne pourrait pas engendrer $1^2, 2^2, 3^2, \dots$
- **2. Si on inverse les deux instructions de la boucle :**

```

1 while s < 1000:
2     n = n + 1
3     s = s + n**2

```

n est augmenté *avant* d'ajouter n^2 : le carré 1^2 n'est alors **jamais ajouté** (la somme commence à 2^2). Le résultat change :

- version *initiale* : la somme atteint 1015 ($= 1^2 + \dots + 14^2$) et **print(n)** affiche **15** ;
- version *inversée* : la somme atteint 1014 ($= 2^2 + \dots + 14^2$) et **print(n)** affiche **14**.

Inverser les deux lignes modifie donc à la fois la somme calculée et la valeur affichée.

Exercice 2 — Le puits

- **1. Coût d'un puits de 10 m.** Le 1^{er} mètre coûte 100 €, puis les mètres suivants coûtent 20, 40, 60, ... € (mètre k : $20(k-1)$ € pour $k \geq 2$). Pour 10 mètres :

$$100 + (20 + 40 + \dots + 180) = 100 + 20 \times (1 + 2 + \dots + 9) = 100 + 20 \times 45 = \mathbf{1\,000\text{ €}}.$$

- **2.a. Valeur de profondeur renvoyée par le script.** Le script part de **prix** = 100 et ajoute 20 à chaque tour *tant que* **prix** <= 700 : **prix** prend les valeurs 100, 120, ..., 700, 720. La boucle s'arrête quand **prix** = 720, et l'on obtient alors **profondeur** = **32**.

- **2.b. Profondeur maximale réelle avec 700 €.** Attention : le script est **trompeur**, car sa variable **prix** n'accumule pas les coûts réels (croissants) — elle se contente d'ajouter 20 € à chaque mètre. Le coût *cumulé* réel de d mètres vaut

$$C(d) = 100 + 10d(d-1).$$

On calcule : $C(8) = 100 + 10 \times 8 \times 7 = 660$ € (≤ 700) et $C(9) = 100 + 10 \times 9 \times 8 = 820$ € (> 700). Le jardinier pourra donc creuser au maximum **8 mètres** (et non 32) avec son budget.

Exercice 3 — Somme de deux dés

► La fonction lance deux dés *tant que* leur somme n'est pas 12 (ce qui n'arrive que si les deux dés affichent 6), et compte le nombre de lancers nécessaires.

► **Tests.** Le résultat **varie à chaque exécution**, par exemple :

```
>>> somme12()
41
>>> somme12()
9
>>> somme12()
57
```

► **Remarque.** La probabilité d'obtenir 12 (double 6) est $\frac{1}{36}$: il faut donc en moyenne **environ 36 lancers**, mais ce nombre fluctue beaucoup d'un essai à l'autre.

Exercice 4 — La fonction mystère

```
1 def mystere(x, y):
2     z = 0
3     while x != 0:
4         if x%2 == 0:
5             x = x // 2
6             y = 2 * y
7         else:
8             x = x - 1
9             z = z + y
10    return z
```

► **2. Tableau complété.**

x	y	z (résultat)
2	5	10
5	15	75
2	-8	-16
100	0,01	1.0
5	2,3	11.5

► **Exemple détaillé : `mystere(5, 15)`.**

Étape	x	y	z
départ	5	15	0
x impair	4	15	15
x pair	2	30	15
x pair	1	60	15
x impair	0	60	75

► **3. Que fait cette fonction ?** Dans tous les cas, $z = x \times y$: la fonction **calcule le produit de x et y** (c'est la « multiplication du paysan russe », par doublements et divisions par 2 successifs).

Exercice 5 — Le distributeur (DAB)

```
1 def DAB(n):
2     if n%10 != 0:
3         return 'impossible'
4     else:
5         i = 0
6         while n >= 20:
7             n, i = n - 20, i + 1
8             billets_20 = i
9             billets_10 = n // 10
10        return billets_10, billets_20
```

► **1. La ligne 2**, `if n%10 != 0`, teste si la somme demandée **n'est pas un multiple de 10**. Comme le distributeur ne possède que des billets de 10 € et 20 €, il ne peut délivrer que des montants multiples de 10 : sinon, il renvoie 'impossible'.

► **2. Retrait de 330 €**. 330 est multiple de 10 : le retrait est possible. La boucle retire des billets de 20 € *tant que* le montant restant est ≥ 20 :

$$330 \rightarrow 310 \rightarrow \dots \rightarrow 30 \rightarrow 10, \quad \text{soit 16 billets de 20 € retirés.}$$

Il reste alors $n = 10$ €, soit $10/10 = 1$ billet de 10 €. La fonction renvoie donc $(\text{billets_10}, \text{billets_20}) = (1, 16)$: **1 billet de 10 € et 16 billets de 20 €**.

► **Vérification** : $16 \times 20 + 1 \times 10 = 320 + 10 = 330$ €, avec le minimum de billets (17 au total).